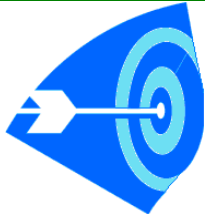


Unidad 6. Constructores y Herencia

Objetivos



Definir e implementar las técnicas de la programación orientada a objeto, mediante la encapsulación, los constructores y la herencia con el lenguaje de programación JAVA:

- ✓ Comparar métodos de objeto y estáticos
- ✓ Utilizar métodos sobrecargados
- ✓ Utilizar la encapsulación para proteger datos
- ✓ Crear constructores para inicializar objetos
- ✓ Definir e implementar el uso de la herencia
- ✓ Explicar la abstracción

Contenido de la unidad

- 6.1 Métodos y variables de objeto y estáticos
- 6.2 Usando la encapsulación
- 6.3 Describiendo el alcance de las variables
- 6.4 Constructores
- 6.5 Implementando la herencia en JAVA

Introducción

La encapsulación se refiere al hecho de “ocultar” una variable de nuestra clase hacia el exterior, de modo que solo pueda ser “vista” o más bien accedida a través de métodos de la propia clase. Otra clase no la podrá acceder, para poder acceder a ella desde otra clase, esta última tendrá que llamar a un método de la clase origen que pueda acceder a ella. Para encapsular un miembro de una clase en su declaración debemos anteponer la palabra reservada “**private**”, siendo privada la variable miembro no podrá ser accedido desde afuera, solo las funciones propias de la clase la podrán manipular.

Los constructores son funciones especiales de la clase, que llevan el mismo nombre de la clase y se utilizan generalmente para inicializar sus datos miembros. Estas funciones no tienen ningún tipo de retorno, pero si pueden tomar parámetros en su llamada, para inicializar todos, algunos o solo un dato de la clase.

Unidad 6. Constructores y Herencia

La herencia hace referencia a que una clase llamada derivada o hija puede heredar los miembros de otra clase, llamada generalmente como clase base o madre. Para establecer una relación de herencia en JAVA se debe especificar la palabra reservada **extends** en la declaración de la clase hija, especificando a continuación el nombre o identificador de la clase de que deseamos heredar. La clase derivada va a poseer todas las variables y métodos de la clase base, excepto los declarados como privados.

El encapsulamiento y la herencia junto al polimorfismo y la abstracción son los pilares principales de la programación orientada a objeto. Aunque ya hemos trabajado con métodos en las unidades anteriores, en esta unidad estaremos trabajando más a conciencia como se desarrollan e implementan los métodos asociados a los principales conceptos de la programación orientada a objeto utilizando el lenguaje de programación JAVA.

6.1 Métodos y variables de objeto y estáticos

Generalmente el código o programa que escribimos en JAVA está definido en los métodos de las clases que escribimos en este lenguaje de programación. Los métodos están pensados en la programación orientada a objeto para que cada uno de ellos trabaje de forma independiente.

La sintaxis en JAVA para definir un método es la siguiente:

```
[modificador] Tipo nombre_metodo ([argumentos])  
{  
  
    // Código del método  
  
};
```

Dónde:

modificador: es el método de acceso al método, ej. public

Tipo: es el tipo de dato que retorna (En el caso de retornar algún valor).

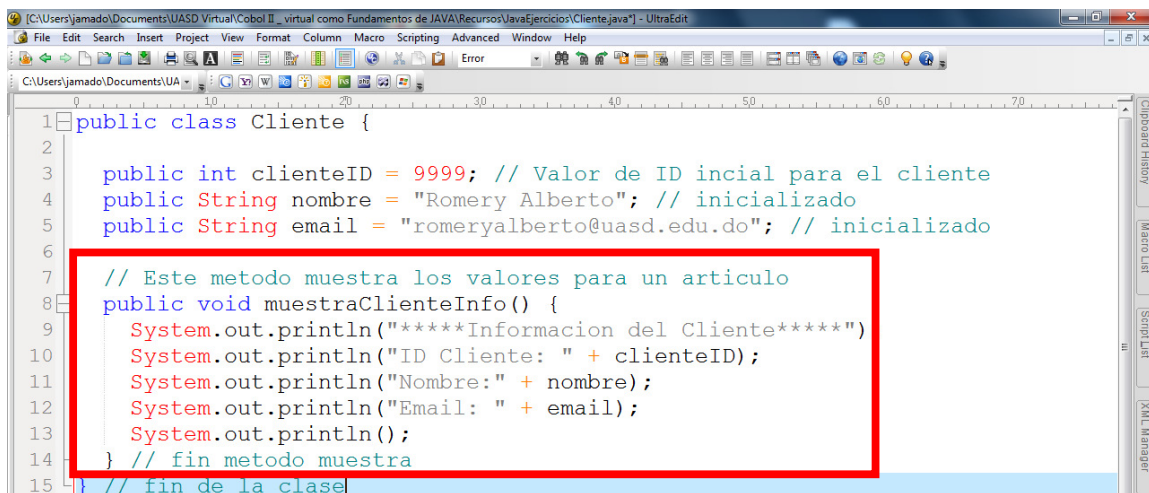
nombre_metodo: es el identificador del método, el nombre que elige el programador para asignarle a su método.

argumentos: son valores con los que se llamará el método. Los parámetros los podemos ver como variables locales al método, pero su valor se inicializa con datos que llegan cuando lo llamamos. Un método puede ser llamado sin argumentos.

Unidad 6. Constructores y Herencia

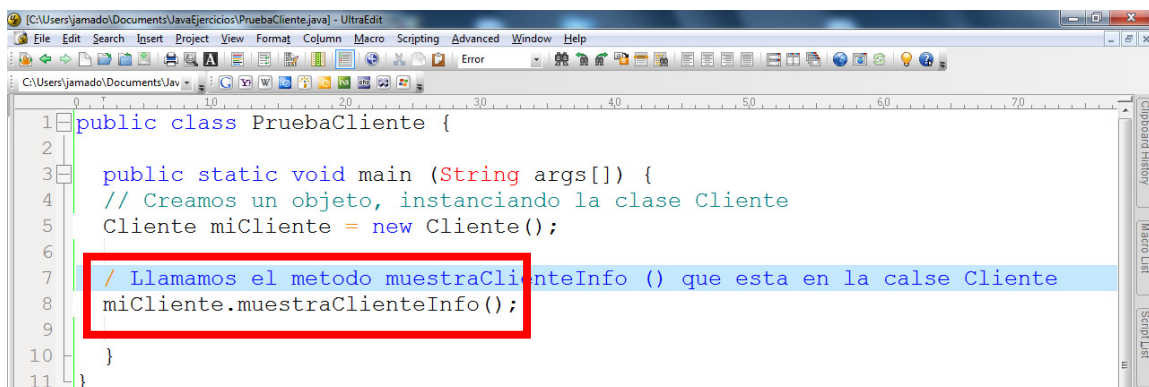
Código del método: Es el código que deseamos programar y ejecutar dentro del método.

Ejemplo: En la siguiente imagen podemos ver la clase cliente que hemos trabajado en unidades anteriores, ahí podrán visualizar el método **muestraClienteInfo()** este método no retorna ninguna valor por lo tanto su tipo es **void** y no se le pasan argumentos por eso, dentro de los paréntesis no escribimos nada.



```
1 public class Cliente {
2
3     public int clienteID = 9999; // Valor de ID inicial para el cliente
4     public String nombre = "Romery Alberto"; // inicializado
5     public String email = "romeryalberto@uasd.edu.do"; // inicializado
6
7     // Este metodo muestra los valores para un articulo
8     public void muestraClienteInfo() {
9         System.out.println("*****Informacion del Cliente*****");
10        System.out.println("ID Cliente: " + clienteID);
11        System.out.println("Nombre: " + nombre);
12        System.out.println("Email: " + email);
13        System.out.println();
14    } // fin metodo muestra
15} // fin de la clase
```

Para invocar o ejecutar un método desde una clase diferente debemos de usar el operador punto (.) con la variable de referencia a objeto que hayamos definido para instanciar la clase en cuestión. Ej:



```
1 public class PruebaCliente {
2
3     public static void main (String args[]) {
4         // Creamos un objeto, instanciando la clase Cliente
5         Cliente miCliente = new Cliente();
6
7         // Llamamos el metodo muestraClienteInfo () que esta en la clase Cliente
8         miCliente.muestraClienteInfo();
9
10    }
11}
```

Para invocar un método en la misma clase solo basta con incluir como instrucción o sentencia el nombre del método, indicando los argumentos correspondientes si existen.

Unidad 6. Constructores y Herencia

Para pasar argumentos y retornar valores en los métodos, es importante tener presente los conceptos fundamentales siguientes:

- ✓ Los métodos pueden ser invocados con una lista de argumentos, los cuales funcionan como datos de entrada o de insumo a dicho método.
- ✓ Los métodos pueden retornar valor que puede ser utilizado posteriormente en el método que originalmente realizó la llamada al método que retornó el valor.

Muchos métodos que son declarados o usados desde la librería estándar de JAVA aceptan argumentos. El método **main** debe ser escrito para aceptar valores cuando el programa es ejecutado desde la línea de comando.

public static void main (String args [])

Los pasos a realizar para invocar o llamar un método con argumentos son:

- ✓ Incluir los argumentos dentro del paréntesis del método que se llama.
- ✓ Pasar valores literales o variables como argumentos.
- ✓ Se debe usar comillas encerrando argumentos que sean valores literales del tipo Strings.
- ✓ Pasar los argumentos requeridos como una lista de parámetros listados en el mismo orden en el cual ellos están definido en la declaración del método que estamos invocando.

Para declarar un método que retorne un valor, se debe colocar el **tipo** del valor que desea que el método devuelva delante del identificador o nombre de método. Para retornar un valor desde un método, se debe usar la palabra reservada **return**. Ej:

```
public int suma (int no1, int no2) {  
    int resultado = no1 + no2;  
    return resultado;  
}
```

Si invocamos un método que retorna un valor, podemos utilizar ese valor retornado dentro de nuestro programa.

Unidad 6. Constructores y Herencia

Ventajas de utilizar métodos	
Los métodos son el mecanismo por el cual los objetos interactúan. En los programas JAVA es común tener múltiples objetos interactuando invocando métodos de otros objetos. Las principales ventajas de utilizar métodos son las siguientes:	
	Los métodos hacen que los programas sean más fáciles de leer y mantener.
	Los métodos hacen posible que el desarrollo y el mantenimiento se realicen de manera más rápida.
	Los métodos son la columna vertebral del concepto de reutilización de código.
	Los métodos permiten separar objetos para comunicar y distribuir el trabajo desarrollado por el programa, haciéndolo más modular.

6.2 Usando la encapsulación

La encapsulación se refiere al ocultamiento de los datos dentro de las clases, esto hace que de una manera fácil para los programadores puedan usar sus clases y proteger ciertos datos dentro de la clase aplicando el modificador apropiado.

La encapsulación en las clases requiere que el programador aplique apropiadamente los modificadores de visibilidad a los elementos en el programa como son la clase, las variables de atributos y la declaración de los métodos.

Los modificadores de visibilidad son dos **public** y **private**. Los atributos y los métodos pueden tener modificadores de visibilidad o acceso que indiquen el nivel de acceso que otros objetos pueden tener sobre ellos.

El modificador **public** permite que la clase, atributos y métodos sean visibles o accedidos por cualquier objeto en su programa. Para hacer que un atributo o método sea público, se debe colocar la palabra reservada **public** delante de la variable de atributo o el método en cuestión. Si todos los atributos en un programa son **public**, sus valores pueden ser cambiando sin ningún error presentarse; esto puede crear problemas durante la ejecución del programa.

Unidad 6. Constructores y Herencia

El modificador de acceso **private** determina sobre los objetos de una clase, sus atributos y métodos (operaciones) ser inaccesibles por otros objetos. Para asignar un atributo o método **private**, se debe colocar la palabra reservada **private** delante de la variable de atributo o método.

Los elementos que tengan como modificador de acceso **private**, tienen las siguientes características:

- ✓ Solo pueden ser modificados o visualizados por los métodos de la propia clase.
- ✓ No pueden ser modificados o visualizados directamente por otra clase.

Los métodos y las variables de atributos declaradas como **public** son frecuentemente denominados como la interface de una clase porque ellos representan los únicos elementos que otra clase puede usar. El detalle de como una clase completa una operación dentro de un método, es llamada implementación del método.

Los objetos pueden acceder a los atributos privados de otro objeto, a través de los métodos públicos de este último objeto.

Después que hayamos escrito una clase encapsulando los datos usando el modificador de acceso **private** y tenemos los métodos **getter** y **setter**, podemos escribir una clase que invoque estos métodos para acceder a los valores de los atributos del objeto.

El código de la encapsulación consiste en declarar los atributos de la clase como **private** y los métodos declararlos como **public**, los cuales serán usados para acceder a las variables de atributos.

6.3 Describiendo el alcance de las variables

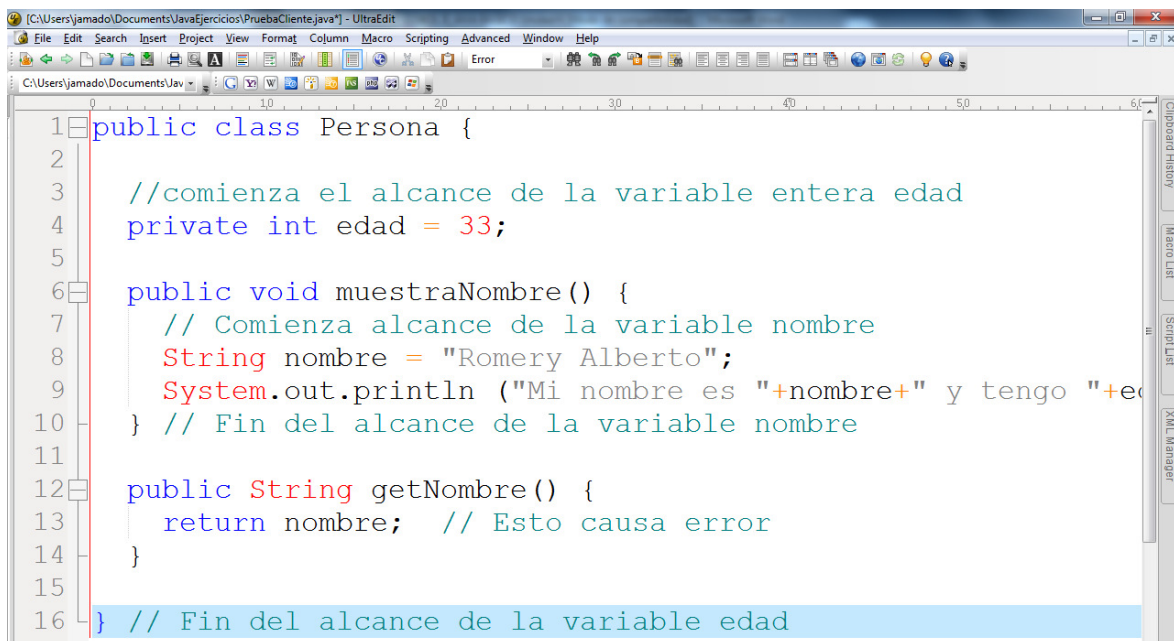
Las variables declaradas dentro de un método, un constructor, o un bloque de código no pueden ser usados en una clase. El alcance de una variable hace referencia al ámbito en el cual una variable puede ser usada dentro de un programa. Los siguientes criterios explican el alcance de una variable:

- ✓ Una variable definida dentro de un método es conocida como **variable local**.

Unidad 6. Constructores y Herencia

- ✓ Una variable local puede solo ser usada dentro del método en el cual este definida y existe solo dentro de los límites de dicho método.
- ✓ Una variable definida fuera de un método en una clase como atributo, existe a lo largo de la vida útil del objeto que se ha creado a partir de dicha clase.
- ✓ Una variable declarada en un **ciclo** o una sentencia condicional **if** es válida solo dentro del ciclo o de la sentencia condicional if.

El código mostrado en la siguiente figura representa el alcance para la variable **edad** y para la variable **nombre**. La variable **edad** está definida fuera de un método en la línea 4 y existe a lo largo de la vida útil del objeto que se cree a partir de la clase **persona**. El alcance de la variable **edad** es a nivel de la **clase**. Por otro lado tenemos la variable **nombre** que es declarada dentro del método **muestraNombre()** en la línea 6, por lo que su alcance y uso está limitado dentro de dicho método. Referirse a la variable **nombre** dentro de otro método, como el **getNombre()** en la línea 12 hasta la 14 produce un error de compilación.



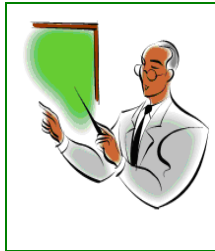
```
1 public class Persona {
2
3     //comienza el alcance de la variable entera edad
4     private int edad = 33;
5
6     public void muestraNombre() {
7         // Comienza alcance de la variable nombre
8         String nombre = "Romery Alberto";
9         System.out.println ("Mi nombre es "+nombre+" y tengo "+edad);
10    } // Fin del alcance de la variable nombre
11
12    public String getNombre() {
13        return nombre; // Esto causa error
14    }
15
16 } // Fin del alcance de la variable edad
```

Como programadores debemos tener siempre presente la habilidad de poder identificar las cuestiones de ámbito de las variables en todo momento, especialmente cuando se utilizan en bucles y sentencias condicionales.

Unidad 6. Constructores y Herencia

6.4 Constructores

Los constructores son métodos especiales de la clase que llevan el mismo nombre de la clase y su función es inicializar los datos miembros (variables de atributo) de la clase. Los constructores son invocados cuando se instancia una clase creando un objeto de la misma.



Como los demás métodos un constructor puede ser sobrecargado, incluyendo varios constructores en una clase, cada cual con el mismo nombre de la clase y una diferente series de argumentos.

Constructor por defecto, el constructor por defecto es creado por el compilador del lenguaje de programación JAVA, si el compilador encuentra en una clase que no se ha definido un constructor explícitamente. El constructor por defecto no aparece explícitamente en el código. Si una clase contiene un constructor definido por el programador el compilador no crea, ni utiliza el constructor por defecto.

Es posible para el programador crear su propio constructor sin argumentos, digitando en el código un constructor sin argumentos para su clase.

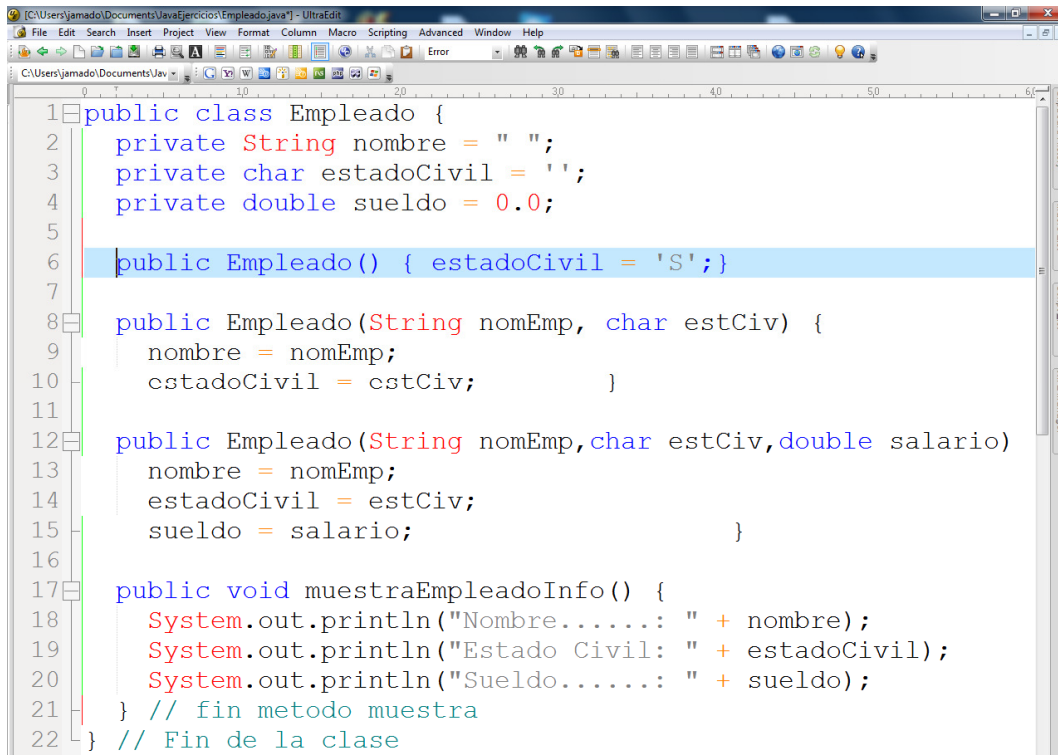
Los constructores pueden ser sobrecargados ofreciendo una variedad de forma para que los objetos sean creados e inicializados usando una clase simple. El siguiente código muestra la representación de tres constructores para la clase empleado.

```
public Empleado() { estadoCivil = 'S';}

public Empleado(String nomEmp, char estCiv) {
    nombre = nomEmp;
    estadoCivil = estCiv;    }

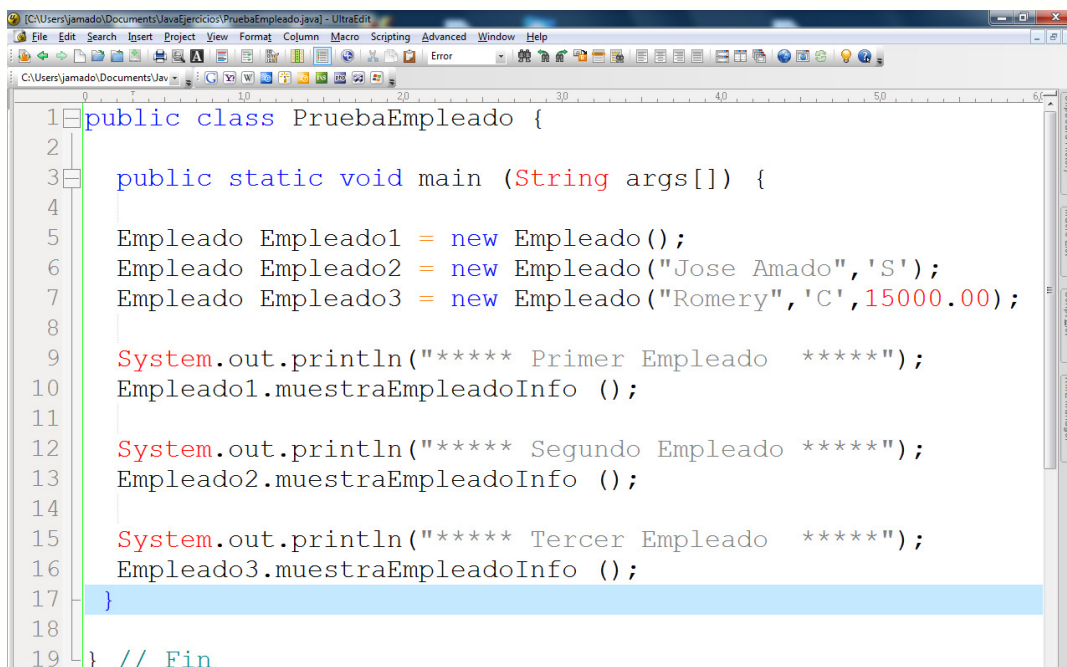
public Empleado(String nomEmp,char estCiv,double salario) {
    nombre = nomEmp;
    estadoCivil = estCiv;
    sueldo = salario;    }
```


Unidad 6. Constructores y Herencia



```
1 public class Empleado {
2     private String nombre = " ";
3     private char estadoCivil = " ";
4     private double sueldo = 0.0;
5
6     public Empleado() { estadoCivil = 'S'; }
7
8     public Empleado(String nomEmp, char estCiv) {
9         nombre = nomEmp;
10        estadoCivil = estCiv;
11    }
12
13    public Empleado(String nomEmp, char estCiv, double salario) {
14        nombre = nomEmp;
15        estadoCivil = estCiv;
16        sueldo = salario;
17    }
18
19    public void muestraEmpleadoInfo() {
20        System.out.println("Nombre.....: " + nombre);
21        System.out.println("Estado Civil: " + estadoCivil);
22        System.out.println("Sueldo.....: " + sueldo);
23    } // fin metodo muestra
24 } // Fin de la clase
```

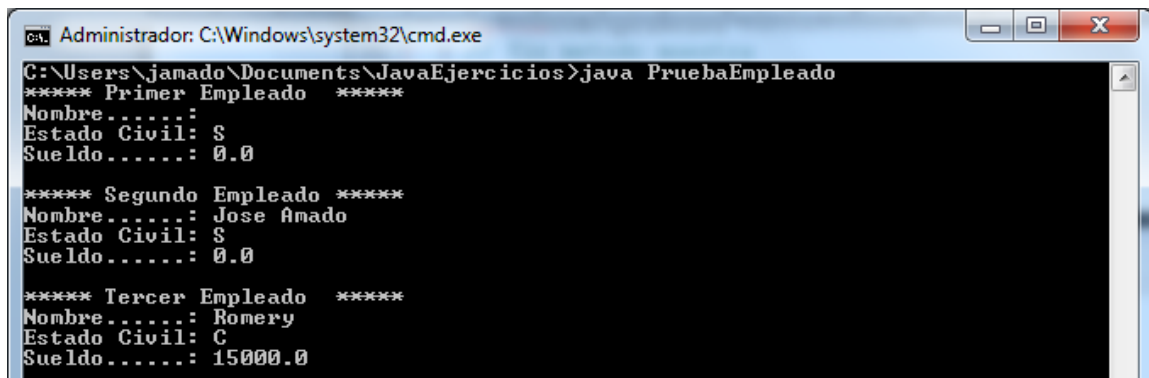
Ejecutado desde la clase **PruebaEmpleado** tenemos:



```
1 public class PruebaEmpleado {
2
3     public static void main (String args[]) {
4
5         Empleado Empleado1 = new Empleado();
6         Empleado Empleado2 = new Empleado("Jose Amado", 'S');
7         Empleado Empleado3 = new Empleado("Romery", 'C', 15000.00);
8
9         System.out.println("***** Primer Empleado *****");
10        Empleado1.muestraEmpleadoInfo ();
11
12        System.out.println("***** Segundo Empleado *****");
13        Empleado2.muestraEmpleadoInfo ();
14
15        System.out.println("***** Tercer Empleado *****");
16        Empleado3.muestraEmpleadoInfo ();
17    }
18
19 } // Fin
```

La salida si ejecutamos el ejemplo anterior seria:

Unidad 6. Constructores y Herencia



```
C:\Users\jamado\Documents\JavaEjercicios>java PruebaEmpleado
***** Primer Empleado *****
Nombre.....:
Estado Civil: S
Sueldo.....: 0.0

***** Segundo Empleado *****
Nombre.....: Jose Amado
Estado Civil: S
Sueldo.....: 0.0

***** Tercer Empleado *****
Nombre.....: Romery
Estado Civil: C
Sueldo.....: 15000.0
```

En el ejemplo anterior tenemos el constructor `Empleado()` sobrecargado, en el cual se crean tres objetos usando los tres constructores de la clase `Empleado`.

6.5 Implementando la herencia en JAVA

La herencia permite crear clases a partir de otras clases, pudiendo los programadores definir una clase con miembros comunes (datos miembros y funciones miembros), que son variables y métodos, en una clase y que otras nuevas clases hereden estos miembros comunes. Resultando de la herencia una jerarquía de clases, donde encontramos una **súperclase** o clase base y otras clases que heredan que se mencionan como **subclases** o clases derivadas, por lo que:

- ✓ Superclase: Esta es la clase base que contiene los elementos comunes a varias clases.
- ✓ Subclase: Es la clase que hereda o se extiende a partir de la superclase.

Las súper-clases también son conocidas como clase padre y las subclases como clases hijas.

En la programación con JAVA cada clase puede heredar los miembros solo de una súper-clase, es decir NO EXISTE lo que en c++ se llama herencia múltiple, donde una clase hija puede heredar de varias clases padre. Por lo tanto, es importante tener en cuenta el mejor uso de la herencia y utilizarlo sólo cuando sea completamente válido.

En la herencia las clases hijas pueden también agregar más miembros o sobrescribir los heredados.

Un **enfoque práctico** para determinar si una clase puede ser diseñada como una clase hija de otra es la **regla**:

Unidad 6. Constructores y Herencia

“es-un | es-una”

Si la frase “Cliente es una persona” tiene sentido, entonces es muy factible que la clase Cliente pueda ser una clase hija de la clase Persona.

Por el contrario si aplicamos esta simple regla a dos clases que no cumplen dicha afirmación, la relación de herencia no será factible. Por ejemplo si tenemos la clase Automóvil y deseamos saber si podemos establecer una herencia con la clase Cliente, tenemos “Un Automóvil es un Cliente” vemos que no, por lo que no es factible que una clase Automóvil herede de una clase Cliente.

Básicamente la herencia define una relación entre las cosas donde una clase hija comparte los miembros comunes con una clase padre. La herencia define entonces una jerarquía de clases, donde una clase hija hereda de una clase padre y además puede agregar nuevos o redefinir alguno de sus elementos.

A continuación se presenta la sintaxis necesaria para definir una relación de herencia, utilizando la palabra reservada **extends** indicamos que una clase heredará de otra clase. La sintaxis para la definición de una subclase o clase hija es como sigue:

[modificador] class nombre_clase extends nombre_superclase

Dónde:

modificador: es el método de acceso a la clase, ej. public

class: es la palabra reservada para definir a las clases.

nombre_clase: es el identificador de la subclase o clase hija, es el nombre que elige el programador para asignarle a su clase hija.

extends: es la palabra reservada con la cual definimos la relación de herencia entre la subclase y la superclase.

Nombre_superclase: es el identificador de la super-clase o clase padre, es el nombre que ha definido el programador cuando definió a la clase padre.

A continuación mostramos un ejemplo para la declaración y uso de la herencia tomando como superclase a **Clothing** y la clase hija o sub clase será **Shirt**. La superclase **Clothing** contiene los atributos y métodos usados que serán heredados por todas las subclases que se extiendan a partir de ella. Cada subclase heredará estos métodos y atributos, en nuestro ejemplo extenderemos la clase **Shirt** la cual heredará de **Clothing**.

Unidad 6. Constructores y Herencia

```
[C:\Users\jamado\Documents\UASD Virtual\UASD II_virtual como Fundamentos de JAVA\Recursos\SL110\SL110\examples\inheritance\Clothing.java] - UltraEdit
File Edit Search Insert Project View Format Column Macro Scripting Advanced Window Help
C:\Users\jamado\Documents\UASD Virtual\UASD II_virtual como Fundamentos de JAVA\Recursos\SL110\SL110\examples\inheritance\Clothing.java 1003608
1 public class Clothing {
2     private int ID = 0; // Default ID for all clothing
3     private String description = "-description required-"; // default
4     private double price = 0.0; // Default price for all clothing
5     private int quantityInStock = 0; // Default quantity for all clothing
6
7     private static int instanceCounter=0; //Static member incremented in constructor to generate uniqueId
8
9     public Clothing() {
10         ID = instanceCounter++;
11     }
12
13     public int getID() {
14         return ID;
15     }
16
17     public void setDescription(String d) {
18         description = d;
19     }
20
21     public String getDescription() {
22         return description;
23     }
24
25     public void setPrice(double p) {
26         price = p;
27     }
28
29     public double getPrice() {
30         return price;
31     }
32
33     public void setQuantityInStock(int q) {
34         quantityInStock = q;
35     }
36
37     public int getQuantityInStock() {
38         return quantityInStock;
39     }
40 } // end of class
```

Esta es la superclase ***Clothing***, todas las clases que se extiendan a partir de ella, es decir que herenden a partir de ella serán subclases, a continuación declaramos la subclase ***Shirt*** a partir de nuestra superclase ***Clothing***.

```
[C:\Users\jamado\Documents\UASD Virtual\UASD II_virtual como Fundamentos de JAVA\Recursos\SL110\SL110\examples\inheritance\Shirt.java] - UltraEdit
File Edit Search Insert Project View Format Column Macro Scripting Advanced Window Help
C:\Users\jamado\Documents\UASD Virtual\UASD II_virtual como Fundamentos de JAVA\Recursos\SL110\SL110\examples\inheritance\Shirt.java 1003608
1 public class Shirt extends Clothing {
2
3     // The color codes are R=Red, B=Blue, G=Green, U=Unset
4     public char colorCode = 'U';
5
6     // This method displays the values for an item
7     public void displayInformation() {
8
9         System.out.println("Shirt ID: " + getID());
10        System.out.println("Shirt description:" + getDescription());
11        System.out.println("Color Code: " + colorCode);
12        System.out.println("Shirt price: " + getPrice());
13        System.out.println("Quantity in stock: " + getQuantityInStock());
14
15    } // end of display method
16 } // end of class
17
```

La clase ***Shirt*** se extiende a partir de la clase ***Clothing*** heredando sus variables y métodos, pero además contiene variables y métodos que son específicos para un objeto de ***Shirt***.

Unidad 6. Constructores y Herencia



Abstracción

Abstracción se refiere a la creación de clases que son generales y no contienen métodos con una aplicación en particular.

Un buen ejemplo de una clase de abstracción es la clase **Artículos**. Un **Artículo** es un concepto abstracto. Por lo general usted no va a una tienda y dice: "Quiero comprar un artículo", ya que puede referirse a cualquier cosa vendida dentro de una tienda.

Sin embargo, todos los artículos pueden tener algunas características similares en el contexto de un sistema de facturación, como por ejemplo un ID (código único de identificación) o un método para mostrar información sobre el artículo.

Las clases que son genéricas no deben ser totalmente definidas, como la clase **artículo**, ya que ellas hacen referencia a elementos abstractos. Las clases que se extienden o heredan de una clase abstracta pueden implementar los métodos vacíos de la clase abstracta con código específico para la subclase. Usted tendrá que pasar más tiempo en su análisis y diseño para garantizar que su solución tiene suficiente abstracción para asegurar la flexibilidad.

Bibliografía



- ✓ **Perez, G. G.** (2008). *Aprendiendo JAVA y Programación orientada a objetos*. Libro de libre descarga, recuperado el 3 Junio 2013 de: <http://www.compunauta.com/forums/linux/programacion/java/ebook.html>
- ✓ **Sun Microsystems, Inc.** (2008). *Fundamentals of the Java™ Language*. Sun Services, Revisión A.